Design Pattern

By Dheeraj Karki

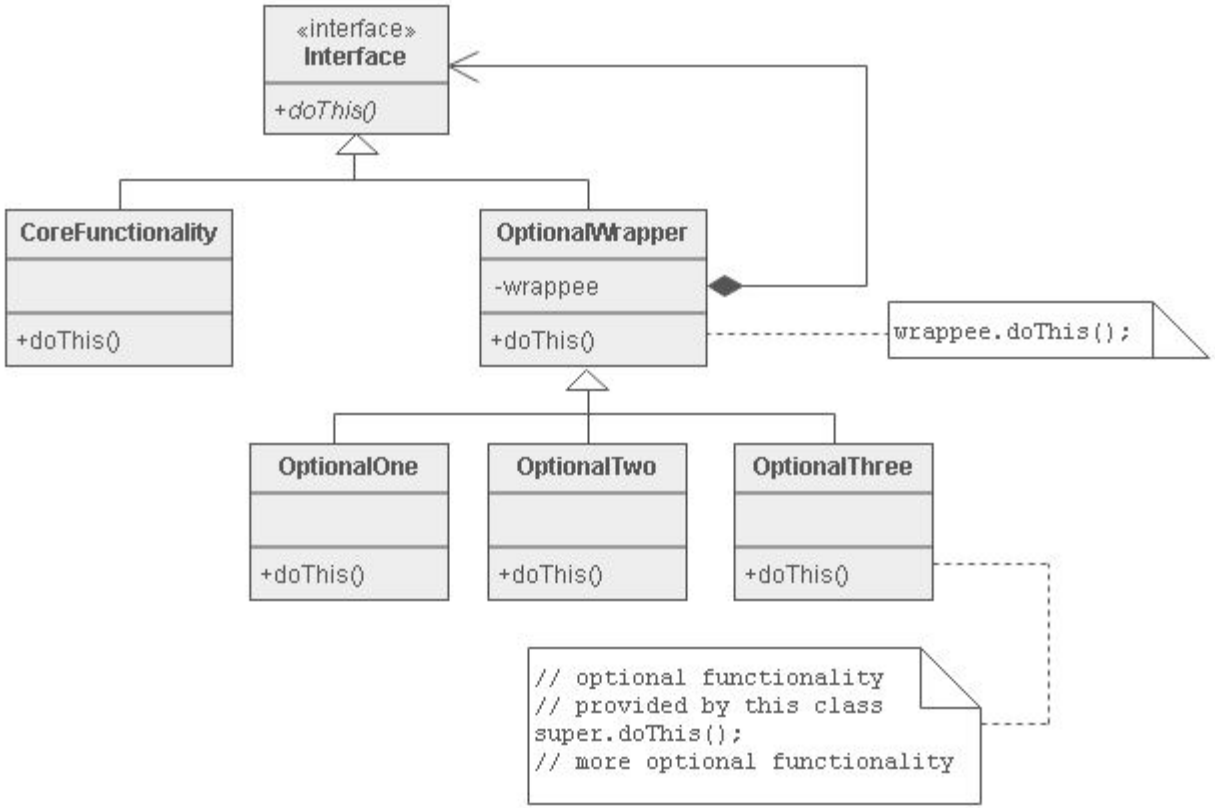
Decorative Pattern

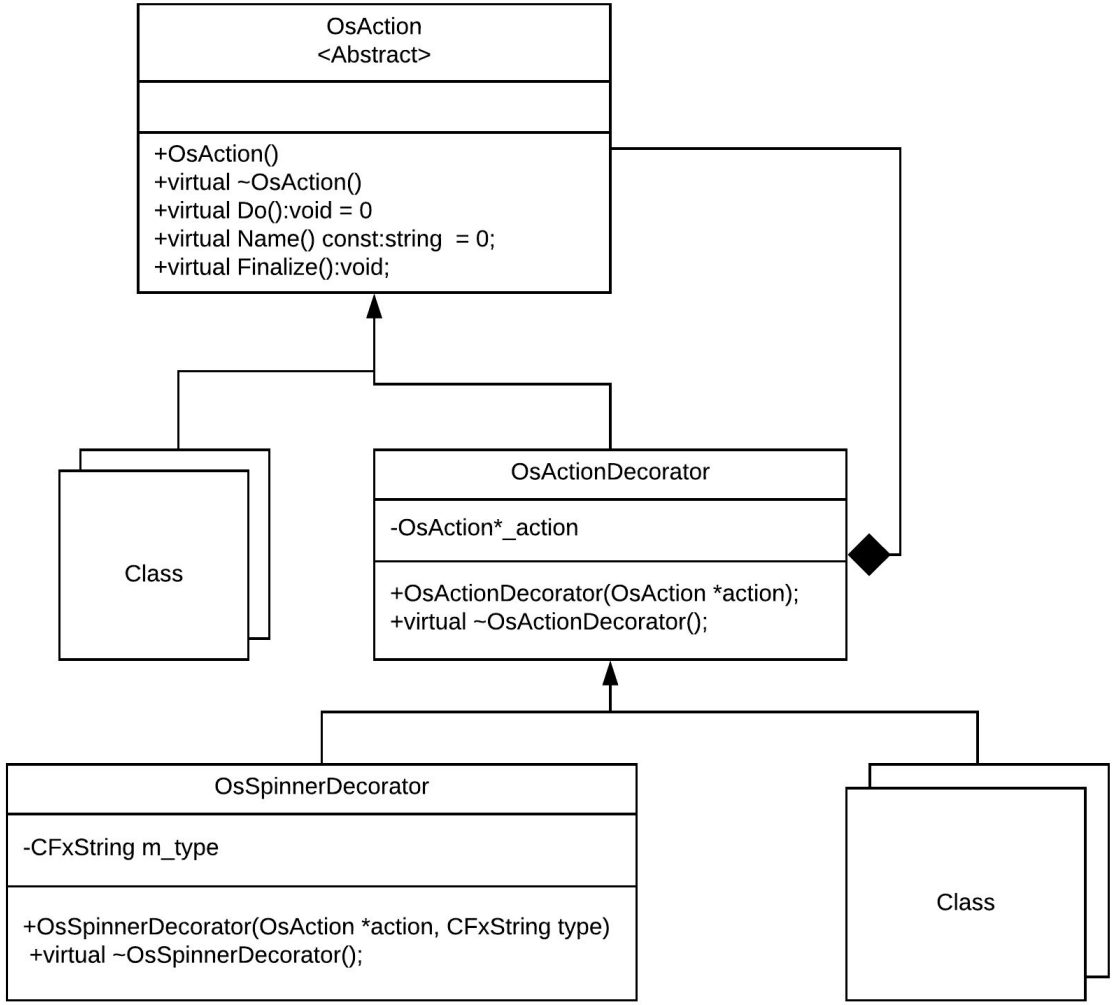
Problem

You want to add behavior or state to individual objects at run-time. Inheritance is not feasible because it is static and applies to an entire class.

Itent

- Attach additional responsibilities to an object dynamically. Decorators provide a flexible alternative to subclassing for extending functionality.
- Client-specified embellishment(the action of adding decorative details.) of a core object by recursively wrapping it.
- Wrapping a gift, putting it in a box, and wrapping the box.


«interface»
Interface
+doThis()
CoreFunctionality
+doThis()
OptionalWrapper
-wrappee
+doThis()
wrappee.doThis();
OptionalOne
+doThis()
OptionalTwo
+doThis()
OptionalThree
+doThis()
// optional functionality
// provided by this class
super.doThis();
// more optional functionality


OsAction
<Abstract>
+OsAction()
+virtual ~OsAction()
+virtual Do():void = 0
+virtual Name() const:string = 0;
+virtual Finalize():void;
Class
OsActionDecorator
-OsAction*_action
+OsActionDecorator(OsAction *action);
+virtual ~OsActionDecorator();
OsSpinnerDecorator
-CFxString m_type
+OsSpinnerDecorator(OsAction *action, CFxString type)
+virtual ~OsSpinnerDecorator();
Class

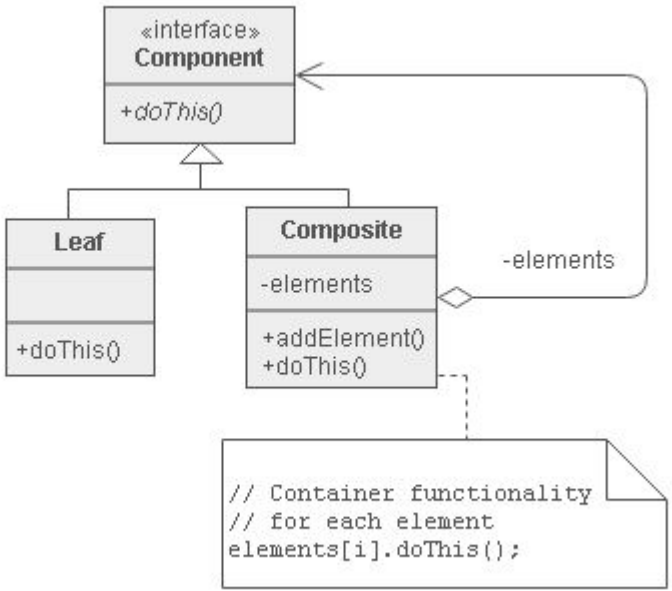
Composite Pattern

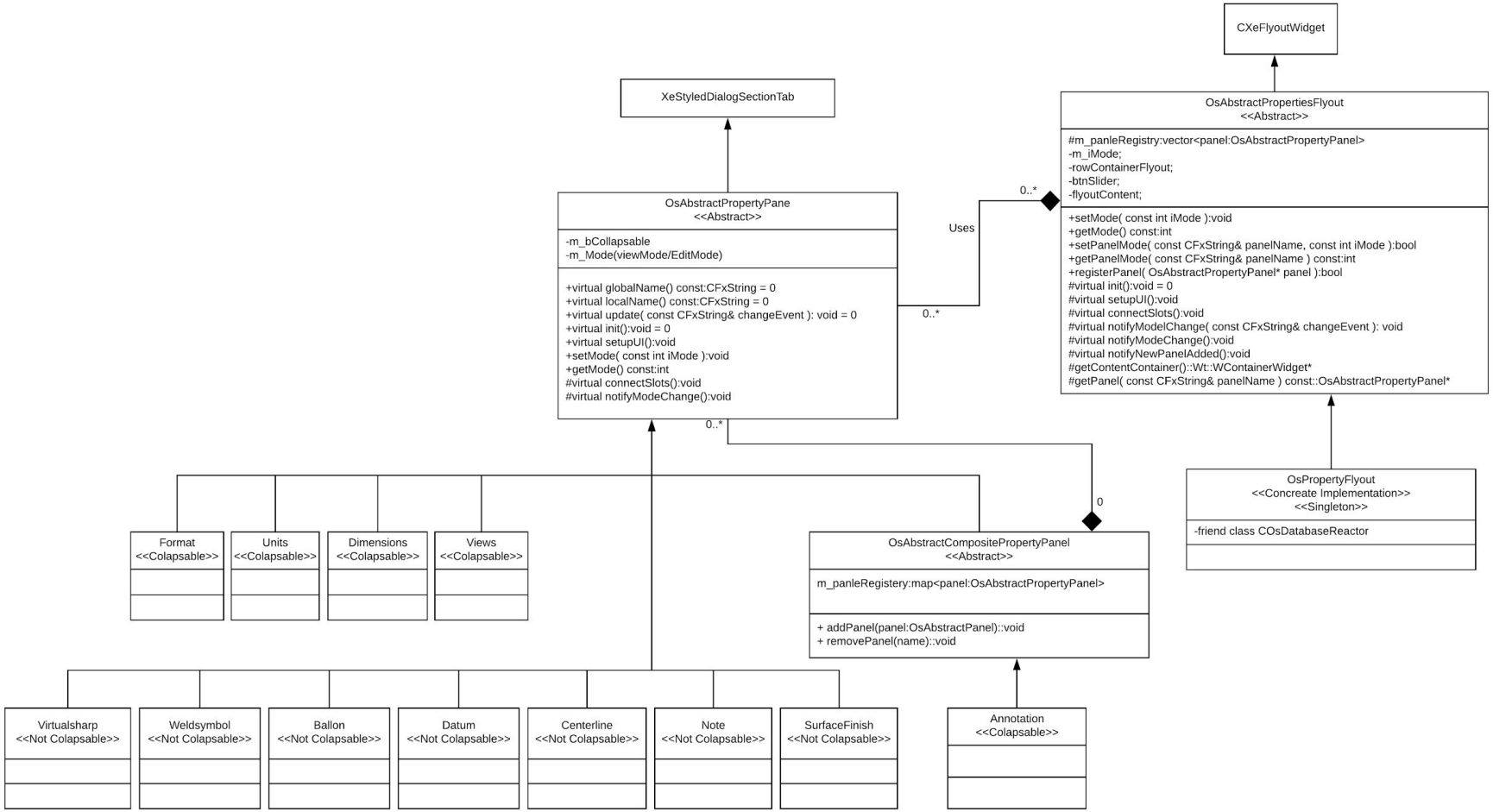
Problem

Application needs to manipulate a hierarchical collection of "primitive" and "composite" objects. Processing of a primitive object is handled one way, and processing of a composite object is handled differently. Having to query the "type" of each object before attempting to process it is not desirable.

Itent

- Compose objects into tree structures to represent whole-part hierarchies. Composite lets clients treat individual objects and compositions of objects uniformly.
- Recursive composition
- "Directories contain entries, each of which could be a directory."
- 1-to-many "has a" up the "is a" hierarchy


«interface»
Component
+doThis()
Leaf
+doThis()
Composite
-elements
+addElement()
+doThis()
-elements
// Container functionality
// for each element
elements[i].doThis();


CXeFlyoutWidget
XeStyledDialogSectionTab
OsAbstractPropertiesFlyout
<<Abstract>>
#m_panleRegistry:vector<panel:OsAbstractPropertyPanel>
-m_iMode;
-rowContainerFlyout;
-btnSlider;
-flyoutContent;
+setMode(const int iMode):void
+getMode() const:int
+setPanelMode(const CFxString& panelName, const int iMode):bool
+getPanelMode(const CFxString& panelName) const:int
+registerPanel(OsAbstractPropertyPanel* panel):bool
#virtual init():void = 0
#virtual setupUI():void
#virtual connectSlots():void
#virtual notifyModelChange(const CFxString& changeEvent): void
#virtual notifyModeChange():void
#virtual notifyNewPanelAdded():void
#getContentContainer()::Wt::WContainerWidget*
#getPanel(const CFxString& panelName) const::OsAbstractPropertyPanel*
0..*
Uses
0..*
OsAbstractPropertyPane
<<Abstract>>
-m_bCollapsable
-m_Mode(viewMode/EditMode)
+virtual globalName() const:CFxString = 0
+virtual localName() const:CFxString = 0
+virtual update(const CFxString& changeEvent): void = 0
+virtual init():void = 0
+virtual setupUI():void
+setMode(const int iMode):void
+getMode() const:int
#virtual connectSlots():void
#virtual notifyModeChange():void
0..*
0
OsPropertyFlyout
<<Concreate Implementation>>
<<Singleton>>
-friend class COsDatabaseReactor
Format
<<Colapsable>>
Units
<<Colapsable>>
Dimensions
<<Colapsable>>
Views
<<Colapsable>>
OsAbstractCompositePropertyPanel
<<Abstract>>
m_panleRegistery:map<panel:OsAbstractPropertyPanel>
+ addPanel(panel:OsAbstractPanel)::void
+ removePanel(name)::void
Virtualsharp
<<Not Colapsable>>
Weldsymbol
<<Not Colapsable>>
Ballon
<<Not Colapsable>>
Datum
<<Not Colapsable>>
Centerline
<<Not Colapsable>>
Note
<<Not Colapsable>>
SurfaceFinish
<<Not Colapsable>>
Annotation
<<Colapsable>>

Thanks
Discussion?